

Rapport d audit DeliverMap

Analyse technique du projet et recommandations pour la partie mobile

Date: 03 juin 2026

Verdict court: non, je ne peux pas dire que tout marche et qu'il n'existe aucun bug. Le frontend se compile correctement en production et le backend passe le contrôle Django, mais plusieurs contrôles automatiques échouent et indiquent des erreurs réelles ou probables dans le code.

Point positif: la base du projet est solide: React/Vite côté frontend, Django REST côté backend, routes nombreuses, WebSocket, tracking, notifications, i18n, modules client/chauffeur/admin et tests backend présents.

1. Vérifications effectuées

Contrôle	Résultat	Conclusion
npm run build	OK	Le frontend produit un build Vite utilisable. Avertissement: bundle JS très gros (~1,96 MB).
npm run lint	ÉCHEC	214 erreurs et 1084 avertissements. Ce n'est pas acceptable pour certifier une version sans bug.
npm run i18n:check	ÉCHEC	Cle manquante: prd_confirm_delete. 870 clés non détectées peuvent être dynamiques.
python manage.py check	OK avec avertissements	Aucune erreur Django critique, mais 18 avertissements sur DEFAULT_AUTO_FIELD.
python -m pytest	NON EXECUTE	Pytest bloque dans cet environnement: ModuleNotFoundError: pygments, malgré une installation présente ailleurs.

2. Bugs et risques détectés

Priorité haute

- **Fonction t non définie** dans plusieurs composants qui importent useI18n sans appeler `const { t } = useI18n()`. Exemples: `frontend/src/components/ClientFormModal.jsx` lignes 25, 35, 41; `frontend/src/components/CommandeFormModal.jsx` lignes 55, 77, 87. Ces composants peuvent planter à l'ouverture.
- **ModelLivraison contient une référence probablement invalide:** `const etapeActuelle = ETAPES[etape]` alors que le tableau défini s'appelle `ETAPES_DEFS`. Fichier: `frontend/src/pages/chauffeur/ModelLivraison.jsx` ligne 303. Risque: écran chauffeur inutilisable.
- **ClientDashboard a du code inaccessible et une fonction manquante:** `setConfirmState` non défini et lignes après `return`. Fichier: `frontend/src/pages/client/ClientDashboard.jsx` lignes 1061-1064. Risque: annulation de commande qui plante.
- **Contrôle i18n en erreur:** la clé `prd_confirm_delete` est utilisée mais absente du dictionnaire FR. Risque: texte affiché brut ou traduction manquante.

Priorité moyenne

- **Lint très bruyant:** 1298 problèmes au total. Beaucoup sont des imports inutilisés, mais ils cachent les vraies erreurs.
- **Bundle frontend lourd:** Vite signale un chunk de 1,96 MB. Sur mobile, cela peut ralentir le chargement et consommer plus de data.
- **Tests backend non confirmés:** les fichiers de tests existent, mais l'environnement local bloque avant exécution. Il faut valider dans Docker ou un venv propre.

- **Encodage de certains fichiers/documentations:** plusieurs textes apparaissent mal encodes dans les sorties console, ce qui peut se voir dans l'interface si les fichiers source sont touchés sans UTF-8 propre.

3. Ce qui semble fonctionner

- Le build production React/Vite se termine avec succès.
- Le backend Django est chargeable et passe `manage.py check` sans erreur bloquante.
- La structure fonctionnelle est riche: client, chauffeur, admin, boutique/store, tracking, notifications, incidents, contrats, scoring, favoris, fidelite.
- Les dictionnaires i18n ont une parité parfaite entre FR, EN, AR et ES, sauf la clé utilisée manquante signalée par le script.

4. Partie mobile: quoi faire concrètement

Objectif recommandé: faire de DeliverMap mobile une expérience chauffeur + client utilisable en situation réelle, pas seulement une interface responsive.

A. Priorité 1 - Stabiliser le mobile existant

- Corriger `ModeLivraison.jsx`: remplacer ETAPES par ETAPES_DEFS, valider toutes les étapes accepter, récupérer, en route, confirmer.
- Corriger toutes les utilisations de `t(...)` sans `useI18n()` dans les composants ouverts depuis mobile.
- Tester les vues sur 360x800, 390x844 et 430x932: login, dashboard client, catalogue, panier, suivi commande, chauffeur dashboard, mode livraison, incident.
- Ajouter un menu mobile clair pour admin/store/chauffeur: navigation bottom bar ou drawer, pas une sidebar desktop réduite.

B. Priorité 2 - Parcours chauffeur mobile

- Faire du `ModeLivraison` l'écran principal chauffeur: gros bouton d'action, adresse, appel client/boutique, navigation Google Maps, incident en un tap.
- Ajouter preuve de livraison réelle: upload photo via camera mobile (`accept=image/* capture=environment`), signature tactile sur canvas, PIN client.
- Ajouter mode hors connexion: conserver mission courante, statut, GPS et photos en local, puis synchroniser quand internet revient.
- Améliorer GPS: permission explicite, état signal faible, bouton recentrer, fréquence adaptée pour ne pas vider la batterie.
- Ajouter checklist sécurité livraison: colis récupérer, client contacté, problème signalé, preuve ajoutée.

C. Priorité 3 - Parcours client mobile

- Optimiser catalogue mobile: recherche sticky, catégories horizontales, panier accessible en bas, fiches produit compactes.
- Suivi live mobile: timeline claire, carte plein écran optionnelle, bouton appeler chauffeur, estimation de temps, notifications de changement de statut.
- Checkout mobile en 3 étapes: adresse, récapitulatif, confirmation. Éviter les longs formulaires sur un seul écran.
- Favoris et historique: cartes compactes, action recommander, filtre par statut.

D. Priorité 4 - PWA et notifications

- Ajouter manifest PWA, icônes, thème color, écran installable, splash propre.
- Ajouter service worker pour cache minimal: assets, shell app, page offline.
- Ajouter push notifications navigateur: commande acceptée, chauffeur proche, livraison terminée, incident.
- Prévoir fallback SMS/email si push non autorisé.

E. Priorité 5 - Performance mobile

- Mettre du code splitting par role: client, chauffeur, admin, store. Le bundle actuel est trop lourd pour une premiere visite mobile.
- Lazy-load cartes Leaflet et pages admin; les cartes sont couteuses et pas necessaires sur tous les ecrans.
- Optimiser images boutique/produit: formats WebP, tailles responsives, skeleton loading.
- Limiter les polling/intervals et fermer les WebSocket au changement de page.

5. Plan de travail recommande

Semaine	Action	Livrable
1	Corriger erreurs lint bloquantes, i18n, ModelLivraison, annulation ClientDashboard.	Application qui passe build + lint critique + i18n check.
2	Refonte navigation mobile chauffeur/client et tests responsive manuels.	Prototype mobile utilisable sur telephone.
3	Preuve de livraison: photo camera, signature, PIN, upload backend.	Flux livraison complet chauffeur.
4	PWA, offline minimal, push notifications, optimisation bundle.	Version mobile presentable et plus rapide.

6. Commandes a relancer avant soutenance

- `cd frontend && npm run build`
- `cd frontend && npm run lint`
- `cd frontend && npm run i18n:check`
- `cd backend && python manage.py check`
- `cd backend && python -m pytest` dans Docker ou un environnement Python propre.

Conclusion

Le projet est avance et riche fonctionnellement, mais il n est pas encore dans un etat ou l on peut promettre zero bug. La priorite doit etre de corriger les erreurs statiques bloquantes, puis de transformer la partie mobile en experience chauffeur/client testee sur vrais formats telephone. Avec ces corrections, DeliverMap peut devenir beaucoup plus credible pour une demonstration ou une soutenance.